

Mantle: Efficient Hierarchical Metadata Management for Cloud Object Storage Services

摘要

云对象存储服务 (Cloud Object Storage Services, COSS) 是云环境中的核心存储后端, 支撑着大规模数据分析与机器学习 (ML) 工作负载 —— 这类负载常需访问深度对象路径, 并对元数据进行并发更新。然而, 当前 COSS 架构存在多轮查找开销高、目录竞争激烈等问题, 导致任务执行延迟增加。此前针对分布式文件系统设计的优化方案 (在云环境中普及率极低), 因 COSS 特有的限制 (如无状态代理、API 功能有限) 而无法直接适用。

Mantle 是一款面向现代云工作负载的新型 COSS 元数据服务, 采用两层架构设计: 其一为可扩展的分片数据库 (TafDB), 供多个命名空间共享; 其二为单命名空间专属的单服务器 IndexNode, 用于整合轻量级目录元数据。通过对元数据与职责的细粒度划分, Mantle 单个命名空间可支持多达 100 亿个对象或目录; 借助 IndexNode 上支持的单 RPC (远程过程调用) 查找可扩展性执行, 其查找吞吐量可达每秒 180 万次。在高竞争场景下, Mantle 通过两项关键优化将目录更新吞吐量提升至每秒 5.8 万次: 一是在 TafDB 中集成异地增量更新 (out-of-place delta updates), 二是将跨目录重命名的循环检测任务卸载至 IndexNode, 两者均有效消除了协同瓶颈。

与 Tectonic、InfiniFS 和 LocoFS 的元数据服务相比, Mantle 将元数据延迟降低 6.6%~99.1%, 吞吐量提升 0.07 倍~115.00 倍。在启用数据访问功能后, 对于交互式 Spark 分析任务, Mantle 将任务完成时间缩短 63.3%~93.3%; 对于 AI 驱动的音频预处理任务, 完成时间缩短 38.5%~47.7%。目前, Mantle 已在百度对象存储 (Baidu Object Storage, BOS) —— 百度沧海存储 (Baidu Canghai Storage) 提供的服务 —— 中部署超过 2 年。

CCS 概念: · 信息系统 → 云存储; 目录结构

1 引言

亚马逊 S3 [5]、微软 Azure Blob 存储 [28] 以及谷歌云存储 [14] 等云对象存储服务 (Cloud Object Storage Services, COSS), 已成为现代云环境中的主流存储基础设施 [41]。它们是各类关键工作负载的基础支撑层, 在数据分析、机器学习 (ML) 训练与推理等数据密集型领域尤为重要。这些应用高度依赖 COSS 存储海量数据集, 并对数据进行高频次、高并发访问。因此, 为保证任务响应效率与成本效益 (尤其是在使用 GPU 等高价计算资源的场景下), 应用对对象路径解析与对象元数据修改的延迟和吞吐量提出了极高要求 —— 需实现超低延迟与高吞吐量。

然而, 在 COSS 中满足这些性能需求并非易事。现代工作负载常需处理深度嵌套的目录层级 (访问路径较长), 且存在大量高并发的目录修改操作。我们对真实世界命名空间的分析 (详见第 3 章) 显示, 这些命名空间规模可达十亿级, 目录平均深度约为 11 层, 最大深度甚至达到 95 层。此外, 多个并发应用会同时访问这些命名空间, 使得单个命名空间的峰值吞吐量可达到每秒数十万次操作甚至更高。在所有元数据请求中, 路径解析 (又称“查找”) 是耗时主力, 在部分场景下占元数据操作总延迟的比例超过 89%。不仅如此, 共享目录的竞争问题会严重降低目录修改操作 (如创建目录 `mkdir`、重命名目录 `dirrename`) 的性能, 进而导致交互式分析查询等应用的性能受限。

尽管已有大量研究聚焦于分布式文件系统（Distributed File Systems, DFS）的元数据优化，但针对 COSS 的相关研究却十分有限——尽管 COSS 在云环境中的普及率远高于分布式文件系统。COSS 架构存在一些根本性限制，例如无状态代理层、API 功能有限、缺乏客户端协同逻辑等，这些限制导致传统 DFS 优化技术无法生效。元数据缓存 [26,42,50]、推测式并行路径解析 [26] 等技术依赖紧密耦合的设计架构，而这正是 COSS 天然缺失的；类似地，通过单分片更新实现的隔离性弱化 [8] 虽能减少目录更新竞争，但无法完全解决 COSS 工作负载中复杂的协同模式，且对路径解析性能的优化效果甚微。

为解决上述局限，本文提出 Mantle——一款新型 COSS 元数据服务，旨在实现超低查找延迟、高吞吐量与抗竞争性能。Mantle 采用包含式两层架构，由可扩展的分片数据库 TafDB 与轻量级单命名空间专属 IndexNode（优化快速目录查找）组成。与此前的分层设计 [21] 不同，Mantle 对元数据与服务职责进行了细粒度划分：TafDB 以规模化方式存储所有元数据，供多个命名空间共享；IndexNode 仅缓存单个命名空间的关键目录元数据，支持高效的单 RPC 路径查找，而非跨节点的多轮路径遍历。

除架构创新外，Mantle 还引入两项关键优化以实现性能目标。首先，Mantle 重新设计了 IndexNode 内部的数据组织与查找服务流程，避免其成为单节点瓶颈。为降低 CPU 开销，Mantle 引入 TopDirPathCache 静态缓存，存储高稳定性路径前缀的解析结果。TopDirPathCache 无需运行时缓存项的晋升或降级操作，而是通过轻量级无锁机制 Invalidator 清除过期缓存项。这种设计在快速查找性能与高效缓存失效（由目录更新触发）之间实现了有效平衡。此外，Mantle 利用 IndexNode 基于 Raft 协议的复制组，将查找任务卸载到从节点（follower）与学习节点（learner），通过负载分发突破主节点（leader）的性能限制，提升查找可扩展性。

其次，为解决并发目录更新中的竞争问题，Mantle 为 TafDB 引入新型增量记录（delta record）机制：以无冲突追加操作替代原地更新，大幅减少高并发场景下的事务中止与重试。针对跨目录重命名等复杂操作，Mantle 将循环检测逻辑迁移至 IndexNode，利用其本地元数据索引通过单次 RPC 完成锁定与验证，避免高昂的分布式协同开销。最后，Mantle 通过 Raft 日志批处理与从节点写卸载机制，确保随着目录更新频率提升，IndexNode 仍能维持较高的写吞吐量。

目前，Mantle 已在百度对象存储（Baidu Object Storage）中部署超过 1.5 年。第 7 章详细介绍了部署经验，包括管理 19 个内部命名空间与数十亿个对象的实践情况。评估结果显示，Mantle 单个命名空间支持 100 亿条记录，查找吞吐量可达每秒 189 万次；在高竞争场景下，创建目录（mkdir）与重命名目录（dirrename）操作的吞吐量分别可达每秒 58.8 千次与 38.0 千次。本文将 Mantle 与 Tectonic [33]、LocoFS [21]、InfiniFS [26] 三款主流分层元数据服务进行对比，基于交互式 Spark 分析、AI 音频预处理两类真实工作负载与 mdtest 基准测试展开评估。结果表明，Mantle 相对上述三款系统的元数据延迟分别降低 17.5%~95.2%、9.5%~99.1%、6.6%~98.8%，吞吐量分别提升 1.20 倍~20.90 倍、1.16 倍~116.00 倍、1.07 倍~80.78 倍。这些性能提升最终转化为业务价值：Spark 分析任务完成时间缩短 63.3%~93.3%，音频预处理任务完成时间缩短 38.5%~47.7%。Mantle 及其分析数据将在<https://mantle-opensource.github.io/> 开源发布。

综上，本文的贡献如下：

1. 对真实世界 COSS 命名空间展开研究，明确由低效查找与高更新竞争导致的关键元数据瓶颈；
2. 提出 Mantle 的两层架构设计：结合单节点目录索引与可扩展数据库，实现元数据批量存储与处理，并通过元数据划分、协同消除等优化避免单节点瓶颈；
3. 实现 Mantle 原型系统，集成上述设计范式，支持十亿级规模的单命名空间；
4. 基于真实应用基准测试与元数据基准测试对 Mantle 进行深度评估，并与当前主流系统对比；
5. 介绍 Mantle 的生产环境部署经验与实践教训，为领域后续研究提供参考。

2 背景

2.1 云对象存储服务（COSS）

据预测，2023 年全球 IT 基础设施支出中，云计算基础设施投资占比将超过 60%[17]。云对象存储服务（COSS）已成为构建云原生应用的核心存储方案。一项针对亚马逊云服务（AWS）上 396 个应用与系统的研究 [41] 显示，约 68% 的应用使用亚马逊 S3 对象存储，而分布式文件系统的重要性较低，使用率仅为 4%。此外，企业存储论坛（Enterprise Storage Forum）报告指出，2020 年全球云对象存储市场规模为 48.3 亿美元，预计到 2028 年将增长至 136.5 亿美元，复合年增长率达 13.6%[40]。COSS 的高普及率与持续扩张的市场规模，凸显了其作为研究方向的重要性与时效性。

上述研究还指出，分析与机器学习（ML）是云应用领域的两大核心工作负载。具体而言，超过 60% 的应用会使用分析或机器学习服务，其中分析类应用占比 42%，机器学习类应用占比 18%。随着人工智能（AI）的快速发展，机器学习工作负载的占比预计将持续提升。COSS 在支撑这两类负载中发挥关键作用：分析引擎通常将预处理与后处理数据存储在 S3 等 COSS 中；而语言处理、视频处理等众多机器学习服务，则依赖 COSS 获取输入数据并存储输出结果。这些结论与我们在百度内部云环境中的观察一致（详见第 3 章）。

2.2 COSS 的接口与编程限制

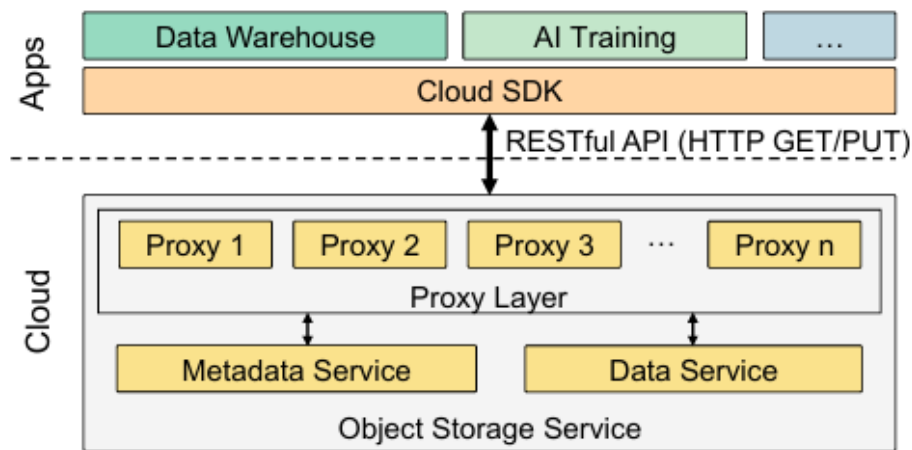


Figure 1. Architecture of typical COSSs with the constrained application interfaces. Note that a single COSS often hosts a number of applications that share the internal metadata and data services.

如图 1 所示，典型的 COSS 由三大核心组件构成：代理层（Proxy Layer）、元数据服务（Metadata Service）与数据服务（Data Service）。其中，数据服务通过一组服务器集群存储对象数据，元数据服务则通过少量服务器管理对象命名空间，实现对象路径到物理数据位置的映射。

通常，多个应用会共享同一个 COSS，它们的命名空间与对象分别存储在 COSS 的元数据服务和数据服务中。COSS 对应用的交互方式存在限制：应用需通过一组有限的 RESTful API [5,28] 与 COSS 交互，且通常通过云 SDK（软件开发工具包）调用这些 API。例如，应用若需获取某个对象，需向随机选择的代理服务器发送包含指定路径的 HTTP GET 请求；代理服务器通过元数据服务解析路径，从数据服务中获取对应对象，最终返回给应用。值得注意的是，代理层采用无状态设计，这一特性有助于实现负载均衡。

COSS 的松散耦合架构将内部组件与外部用户严格隔离，这为元数据服务与数据服务的扩展带来了新的挑战。

2.3 COSS 中的元数据管理

元数据性能对 COSS 应用至关重要。如图 1 所示的“元数据 - 数据分离”架构，对象操作前必须先访问元数据服务；此外，应用中的对象通常体积较小 [3,44]，元数据访问开销在整体对象访问中的占比已不可忽视。后续第 6.2 节将证明，元数据性能的优化可显著提升应用层性能。

近年来，包括亚马逊 S3、微软 Azure Data Lake Storage、谷歌云存储在内的众多 COSS，已逐步增强元数据服务能力——从原本的扁平命名空间管理，过渡到支持分层命名空间 [5,14,28]。此外，这些服务开始采用数据库存储分层命名空间（将命名空间映射为数据库表），通过表分片将数据分布到多台服务器以实现横向扩展，并将元数据操作编排为数据库事务 [7,22,30,33]。上述研究 [41] 还提到，有 16 个应用会在 DynamoDB [9] 中存储指向 S3 数据的指针。

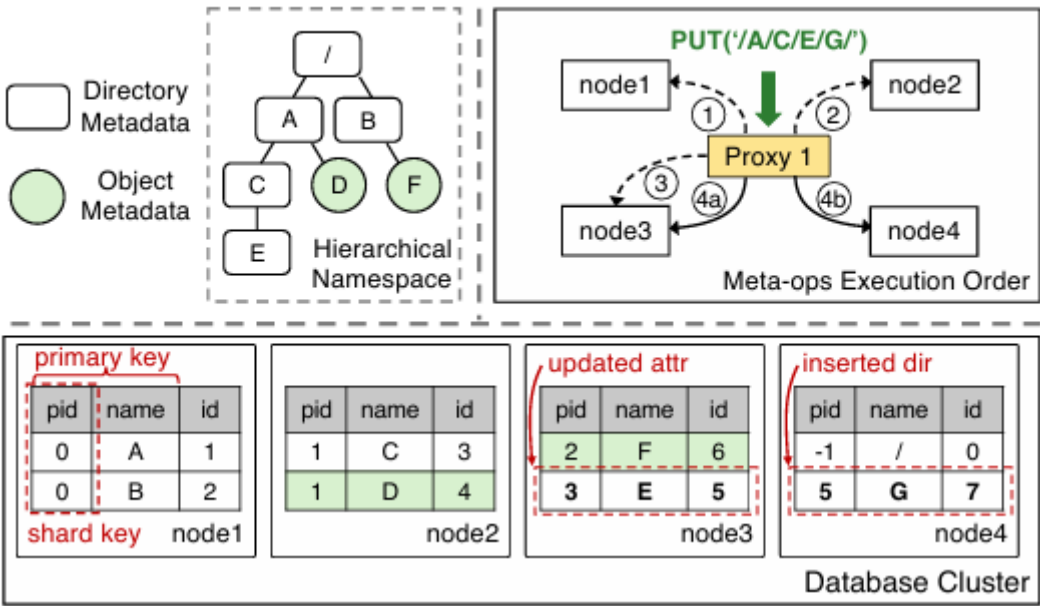


Figure 2. A DBtable-based hierarchical, COSS metadata service. The top left part represents an exemplified namespace containing both directories and objects, with object metadata colored as green. The bottom part depicts the same namespace represented as a table with four shards distributed across four nodes. The top right box shows the execution flow of a mkdir request, where dotted arrows stand for retrieving during path resolution, while solid arrows indicate the insertion of new directory metadata plus the mutation on its parent directory’s metadata, spanning node3 and node4.

图 2 展示了百度 COSS 基于数据库表 (DBtable) 的分层元数据服务架构，该架构与 Meta 公司 Tectonic [33] 存储所采用的架构类似。这种基于 DBtable 的元数据服务，将每个分层命名空间映射为一张大型数据库表，并以父目录唯一标识 (pid) 与对象 / 目录名称作为主键。为在保证目录局部性的同时实现负载均衡，元数据表进一步按 pid 分区，尽可能将同一目录下的表项存储在同一个分片中。

多 RPC 路径解析

在该架构下，解析对象或目录的路径需从根目录开始逐层遍历，代理层与元数据服务（即底层数据库服务器）之间需通过多轮 RPC 交互实现。这一过程源于元数据表的分区特性——每个目录存储在特定的数据库服务器上。在每一层级，代理需先获取当前目录的索引节点 ID（inode ID），才能确定下一层级查找应访问的服务器；同时，每一步都需执行权限检查以确保访问控制合规。如图 2 右上角所示，当应用尝试创建新目录“/A/C/E/G”时，首先向代理层发送 HTTP PUT 请求；随机选中的代理服务器通过从根目录开始递归遍历路径处理该请求，过程中验证每个路径组件的存在性并检查权限（步骤 1、2、3）。

分布式元数据事务处理

创建目录（mkdir）、重命名目录（dirrename）等目录修改操作，需更新多个目录的元数据项。当这些更新涉及多个元数据分片时，需通过分布式事务确保涉及的数据库服务器之间数据强一致 [26,49]。如图 2 所示，步骤 4a 与 4b 共同构成一个由代理发起的分布式事务，用于完成上述 mkdir 请求的后半程操作：该事务协调 node3 与 node4 两台服务器，插入新创建目录“G”的元数据项，并相应更新其父目录“E”的元数据。

3 云对象存储服务（COSS）的命名空间行为

为明确 COSS 上层应用面临的元数据瓶颈问题，我们对百度云环境中数据密集型应用对应的命名空间展开了详细分析——该环境中基于数据库表（DBtable）的元数据服务已部署多年。我们选取了 5 个具有代表性的命名空间（主要承载大数据分析或机器学习任务）进行性能剖析，结果总结于图 3 与图 4 中。

图 3a 显示，这 5 个命名空间均包含超过 20 亿条记录，其中对象占比 82.0%~91.7%，目录占比仅 8.3%~18.0%。此外，这些命名空间的工作负载具有“高频操作大量小对象”的特征，且需维持每秒 15 万~30 万次操作的高吞吐量。得益于固态硬盘（SSD）对小对象数据访问的加速作用，数据访问延迟可降至“单次远程过程调用（RPC）+ 数十微秒设备访问”的水平 [1,4]。在此背景下，元数据访问已成为影响端到端性能的关键因素。

以自动驾驶模型数据预处理场景为例：多个任务需处理包含超过 10 亿个小对象（多数小于 512KB）的大型数据集，每个对象封装了车辆运行过程中采集的短时间片段多模态传感器数据及相关元数据。预处理流程涉及扫描现有对象与创建新对象，会产生大量查找与创建操作负载。在这类场景中，元数据访问占总执行时间的比例超过 70%。

然而，要实现高性能元数据管理面临诸多挑战，核心源于以下两方面效率问题。

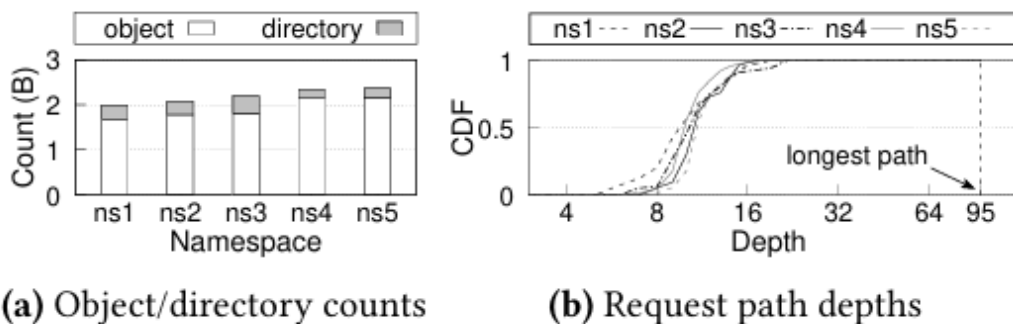


Figure 3. Characteristics of five real-world namespaces

3.1 查找效率低下

令人意外的是，这些命名空间存在深度嵌套的目录层级——路径平均深度超过 10 层，最大深度甚至可达 95 层。且应用的访问模式高度偏向层级结构的深层目录。图 3b 显示，5 个命名空间的平均访问深度分别为 11.6、11.5、10.8、10.6 和 11.9 层；其中规模较大的命名空间 ns4，有一半的请求访问深度超过 10 层的路径。

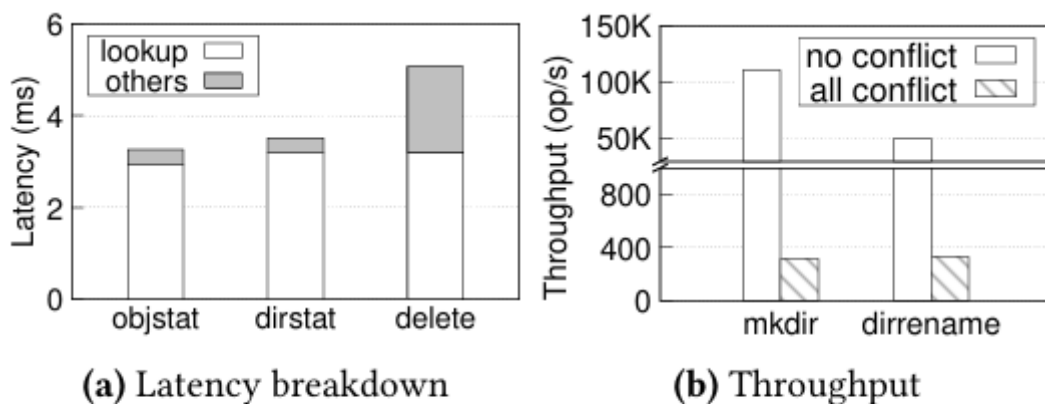


Figure 4. Performance analysis of the DBtable-based COSS metadata service

现代元数据管理方案因“深度目录层级 + 长访问路径”面临路径解析瓶颈。为验证这一点，我们基于 mdtest 基准测试工具 [39]、DBtable 元数据服务与 512 个应用线程（详细配置见第 6.3 节），对对象状态查询 (objstat)、目录状态查询 (dirstat)、删除 (delete) * 这三类元数据操作的延迟构成进行了分析。图 4a 显示，这三类操作的执行时间中，查找（即路径解析）步骤占比最高，分别达到 89.9%、91.2% 和 63.1%。查找耗时较长的原因，与第 2.3 节所述一致——代理层与元数据服务之间需进行多轮交互。需说明的是，该结论同样适用于其他所有元数据操作（因篇幅限制未全部展示），因为查找是所有元数据操作的前置步骤。

* 注：为保持一致性与表述清晰，本文保留 mdtest 工具中定义的操作名称。

3.2 目录更新竞争

创建目录 (mkdir)、删除目录 (rmdir)、重命名目录 (dirrename) 等目录修改操作，在命名空间总访问量中占比相对较低。但对于部分应用而言，由于多个操作会并发更新同一目录的属性，这些修改操作会导致共享目录产生严重的竞争问题。

在我们的生产环境中，Spark 即席查询 (ad-hoc queries) 每天执行超过 1 万次，且对延迟有严格要求。每个查询会生成数百个子任务，这些子任务的临时目录需以原子方式重命名至一个共享输出目录，这一过程会引发集中式的元数据更新。在现有基于 DBtable 的机制下，这种竞争会导致事务频繁重试，使得提交阶段成为整个任务的主要瓶颈；在极端情况下，该阶段耗时占任务总执行时间的比例甚至超过 99%。

为量化共享目录竞争的影响，我们基于 512 个线程运行 mdtest 的 mkdir 与 dirrename 工作负载，并设置两种实验场景：“无冲突”（所有线程写入不同目录）与“全冲突”（模拟上述 Spark 查询场景，所有线程写入同一个目录）。图 4b 显示，在全冲突场景下，基于 DBtable 的元数据服务性能显著低于无冲突场景——两类元数据操作的吞吐量分别下降 99.7% 和 99.4%。性能下降的原因在于：mkdir 与 dirrename 操作会触发跨数据库分片的分布式两阶段提交 (2PC) 事务，在高竞争场景下，这会导致事务中止率升高或锁等待时间延长。

3.3 研究动机

本文旨在为现代 COSS 构建一款新型元数据服务，实现超低查找延迟、降低目录更新竞争，并维持高整体元数据吞吐量。然而，正如表 1 所总结的，现有元数据优化方案大多针对分布式文件系统（DFS），由于架构存在根本性差异，这些方案无法直接应用于 COSS。

方案	适用场景	查找所需往返次数 (RTTs)	目录竞争缓解效果
元数据缓存 [26,42,50]	分布式文件系统 (DFS)	路径长度 (pathlen)	不适用 (N/A)
并行解析 [26]	分布式文件系统 (DFS)	1~ 路径长度 (pathlen)	不适用 (N/A)
隔离性弱化 [49]	分布式文件系统 (DFS)	路径长度 (pathlen)	有限 (restricted)
分层架构 [21]	分布式文件系统 (DFS)	1 次	不适用 (N/A)
Mantle (本文方案)	云对象存储服务 (COSS)	1 次	完全缓解 (all)

表 1 元数据服务方案对比：查找延迟优化与目录修改竞争处理效果。其中，pathlen 表示待解析路径的长度。

4 Mantle 概述

图 5 展示了 Mantle 的高层架构，它是一款专为云对象存储服务（COSS）设计的新型元数据服务，可支持单个命名空间包含多达 100 亿个目录与对象。Mantle 能实现每秒 180 万次路径查找与每秒 3 万次目录层级修改操作，同时大幅降低在该命名空间上运行的云工作负载的延迟。此外，通过加速路径查找，Mantle 还能提升涉及目录与对象的所有其他元数据操作的性能。

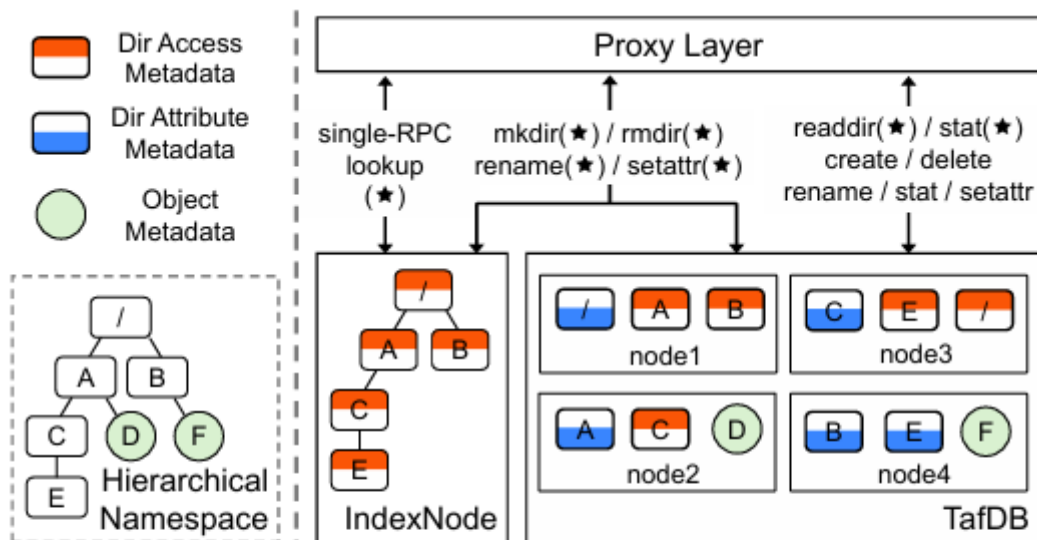


Figure 5. Architecture of the Mantle metadata service and division of metadata request processing over IndexNode and TafDB. Operations marked with (★) are directory operations, while the others are object operations.

为实现上述能力，Mantle 由三大核心组件构成：代理层（Proxy Layer）、TafDB 数据库与 IndexNode 节点。与基于数据库表（DBtable）的元数据服务（图 1）类似，Mantle 的代理层接收应用发送的 HTTP 请求，并通过与内部组件交互协调请求执行，以处理元数据与数据两类请求。其中，TafDB 是一款可扩展的分片数据库，负责存储任意规模的元数据；IndexNode 则是专用的单节点索引，用于整合某个命名空间的全部或部分目录元数据。每个命名空间都会分配专属的 IndexNode，而共享的 TafDB 则作为所有命名空间的可扩展后端。本文重点优化单个命名空间内的元数据性能，但该架构通过部署多个 IndexNode 实例，可自然支持多命名空间部署场景。

与上述分层架构不同，Mantle 在 IndexNode 与 TafDB 之间采用“包含式两层架构”，并对元数据进行细粒度划分。TafDB 保留完整的元数据，而 IndexNode 仅维护目录元数据中的关键子集（每个目录约 80 字节），以实现高效查找与跨目录协同。我们将目录元数据划分为访问元数据（图 5 中红色顶部框）与属性元数据（图 5 中蓝色底部框），其中 IndexNode 仅存储访问元数据。如图 6 所示，这一设计通过 IndexNode 中的 IndexTable 实现，该表包含以下字段：父目录 ID（pid）、目录名称（dirname）、目录 ID（id）、访问权限（permission），以及用于并发更新协同的锁位（lock bit）。

为配合数据划分方案，我们设计了细粒度的职责分工机制：在最大限度降低 IndexNode 工作负载的同时，充分发挥 TafDB 在目录操作方面的横向扩展能力。如图 5 所示，IndexNode 通过单 RPC 操作处理查找与权限检查；TafDB 则利用其存储的属性元数据，处理对象元数据与目录操作（readdir 与 stat）。对于创建目录（mkdir）、重命名（rename）等复杂修改操作，需两个组件协同完成：TafDB 更新所有元数据（访问元数据与属性元数据），IndexNode 则刷新访问数据以支持后续查找，从而维持强同步性。此外，IndexNode 还负责重命名循环检测——对于 TafDB 而言，当目录跨多个数据库分片存储时，该任务难度极高。

IndexNode 的引入并未改变 MetaTable 的可用性保障（由底层数据库确保），因此我们的重点在于确保 IndexNode 自身的高可用性。这一目标通过采用 Raft 共识协议 [31] 实现——将更新复制到多个节点。在 Raft 组内，所有节点均维护相同的内存数据结构，每个节点会根据第 5.1 节所述流程独立构建这些数据结构。

尽管上述设计为元数据管理提供了可扩展性，但也带来了特定挑战——需解决这些挑战以确保系统整体的性能与一致性。下文将详细阐述这些挑战及对应的解决方案。

5 Mantle 的设计

5.1 提升单 RPC 查找的可扩展性

尽管 IndexNode 将分布式查找集中到单个节点处理，但 CPU 密集型的路径解析过程仍会导致 IndexNode 出现性能瓶颈。这是因为 IndexNode 上的单 RPC 查找仍需分解为对 IndexTable 的多次本地访问，且路径深度越深，该瓶颈问题越突出。为此，我们提出了新型 TopDirPathCache 设计，以有效降低 IndexNode 的 CPU 开销（详见 5.1.1 节）；随后，为在低开销前提下维持缓存一致性，我们设计了轻量级缓存失效机制，最大限度减少对查找请求的干扰（详见 5.1.2 节）；最后，在 IndexNode 的 Raft 组中，从节点（follower）通常仅执行复制任务而处于空闲状态，造成 CPU 资源浪费。因此，能否利用从节点分担主节点（leader）的路径解析压力，成为查找优化的另一关键问题，具体设计将在 5.1.3 节详细说明。

5.1.1 TopDirPathCache 缓存

我们在 IndexNode 中设计了名为 TopDirPathCache 的查找缓存，通过存储路径解析结果来减轻 IndexTable 的负载。缓存效率需在内存消耗、性能提升与缓存维护开销之间进行根本性权衡：缓存所有路径会因内存需求过高而不可行（例如，10 亿个目录项需占用超过 1TB 内存）；此外，维持缓存一致性会产生显著开销——祖先目录的任何修改都需失效相关缓存项，给 IndexNode 带来额外 CPU 负担。因此，我们优先缓存稳定性极高的路径解析结果。

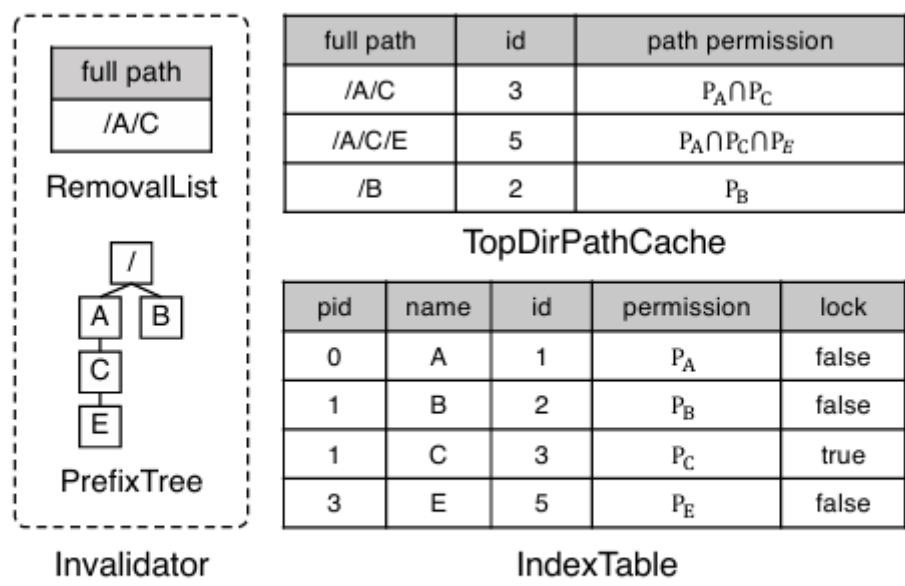


Figure 6. Main data structures on Mantle's IndexNode

对生产环境追踪数据的实证分析表明，大多数目录重命名操作发生在命名空间层级的叶节点附近。基于这一发现，我们决定不缓存完整路径的解析结果，而是仅缓存路径的截断前缀（与叶节点保持一定距离），以此最小化内存占用与缓存晋升 / 降级等频繁维护操作的开销。如图 6 所示，TopDirPathCache 采用内存哈希表实现，每个表项将完整路径前缀映射到对应目录的父目录 ID（pid）及其聚合权限掩码。权限计算采用 Lazy-Hybrid 方法 [6]，通过对路径上所有权限进行交集运算，得到统一的路径权限。

该缓存的工作流程如下：当处理深度为 N 的查找请求时，IndexNode 会截断路径的最后 k 层，并使用得到的前缀查询 TopDirPathCache。其中，经验常数 k 代表“禁用缓存的叶节点距离”——所有距离叶节点 k 层以内的路径均不进入 TopDirPathCache。例如，当解析路径 `/A/C/E/G/H` 且 $k=3$ 时，会检查 TopDirPathCache 中是否存在前缀 `/A/C/`；若未找到对应表项，则会通过 IndexTable 完成路径解析，并为该前缀创建新缓存项。第 6.5 节的评估结果表明， $k=3$ 能在性能提升与内存消耗之间实现平衡：该配置在带来显著性能加速的同时，内存占用仅为“缓存所有结果”方案的 12%。

5.1.2 缓解查找与修改的冲突

目录重命名（dirrename）、属性设置（setattr）等目录修改操作，可能导致相关缓存项失效 —— 所有以受影响路径为前缀的缓存项均需从 TopDirPathCache 中删除。这一过程需要通过范围扫描定位受影响目录的所有后代，但 TopDirPathCache 底层的哈希表并不原生支持范围扫描；此外，还需设计协同机制防止新查找请求访问受影响范围，以保障一致性。

为协调查找与目录修改操作，Mantle 引入了轻量级缓存失效机制 Invalidator，该机制依赖前缀树（PrefixTree）与移除列表（RemovalList）两个辅助数据结构实现。其中，PrefixTree 是无锁基数树，通过重建 TopDirPathCache 中所有缓存路径的目录树，支持高效的失效范围查询；RemovalList 是无锁跳表，用于记录正在被修改的目录的完整路径。这两个数据结构的构建方式如下：当新的查找结果存入 TopDirPathCache 时，PrefixTree 会同步更新以反映新路径；当发生目录修改（如 dirrename、setattr）时，目标目录的完整路径会被插入 RemovalList。Invalidator 中的后台线程会定期轮询 RemovalList，查询 PrefixTree 以获取受影响路径的范围，并从 TopDirPathCache 中删除对应项。对于目录删除（rmdir）操作，由于空目录无法成为现有目录的前缀，不会产生插入冲突，因此无需更新 RemovalList 即可安全删除目标目录。Invalidator 的无锁设计降低了一致性维护的 CPU 开销，其非阻塞架构也最大限度减少了查找操作的竞争 —— 即使在高目录修改负载下，IndexNode 仍能维持高路径解析吞吐量。

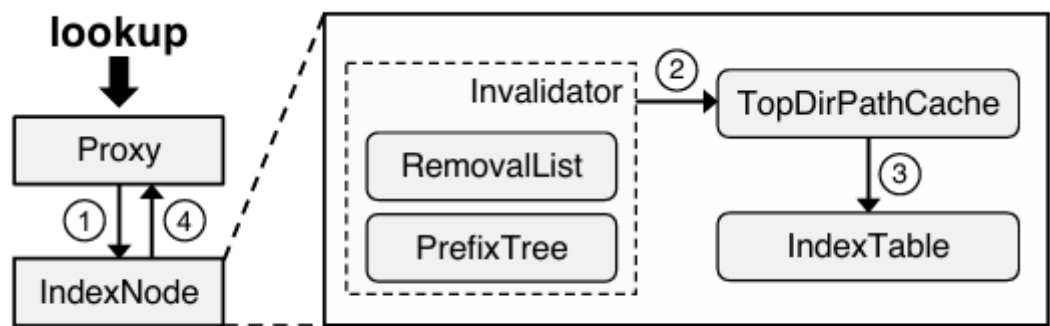


Figure 7. The workflows for processing lookup operation

IndexNode 上的新查找流程涉及多个数据结构的交互，具体如图 7 所示：

1. 查找请求首先扫描 RemovalList，检查是否存在“正在修改且为当前请求路径前缀”的路径。该扫描操作开销极小（多数情况下 RemovalList 为空）；
2. 若存在此类路径，则跳过 TopDirPathCache，直接查询 IndexTable 以避免访问失效缓存（步骤 3）；若不存在，则查询 TopDirPathCache 中的截断前缀路径（步骤 2）；
3. 若前缀已缓存，则通过 IndexTable 逐层遍历剩余的后代目录完成解析（步骤 3）；
4. 查找完成后，若满足以下两个条件，会将截断前缀的解析结果存入 TopDirPathCache：（a）该截断前缀尚未缓存；（b）查找请求执行期间 RemovalList 未被修改。为检测并发修改与查找的冲突，系统采用常规时间戳机制；
5. 最终将请求路径对应的父目录 ID（pid）返回给代理（步骤 4）。

5.1.3 从节点 / 学习节点上的路径解析

为突破单节点资源限制，当主节点负载较高时，我们会将路径解析请求卸载到空闲的 IndexNode 从节点。从节点首先会向主节点查询最新的提交索引（commitIndex），以确保一致性并避免脏读；随后等待本地应用索引（applyIndex）超过 commitIndex 后，再执行本地路径解析流程。为最大限度降低对主节点的开销，对 commitIndex 的查询采用批处理方式。此外，我们还在 Raft 组中加入学习节点（learner，即只读副本）[20]，进一步提升吞吐量 —— 从节点与学习节点均会独立维护并使用本地 TopDirPathCache 加速路径解析。

然而，从节点读取可能面临本地 TopDirPathCache 中存在失效项的风险。为解决这一问题，Mantle 通过 Raft 日志复制失效信息，实现 Raft 组内的缓存失效同步：需执行缓存失效的操作会将受影响目录的完整路径追加到 Raft 日志中；当这些日志被应用时，会通过前文所述的 Invalidator 机制失效对应缓存项。该方案确保从节点与学习节点能及时同步目录修改，且不会产生过高开销。

5.2 提升目录修改的可扩展性

要提升目录修改性能，需重点解决以下三类问题：

1. 如 3.2 节所述，对同一目录的并发修改（尤其是属性元数据更新）会导致事务频繁中止与重试，造成长尾延迟。为缓解该问题，我们引入异地更新机制，消除属性竞争（详见 5.2.1 节）；
2. 优化最复杂的元数据操作——跨目录重命名（dirrename）：利用 IndexNode 将循环检测从分布式协同任务转化为本地操作（详见 5.2.2 节）；
3. 随着目录修改效率显著提升，需确保 IndexNode 能承受更高的写负载。为此，我们设计针对性机制，避免单 Raft 组的容量限制导致写吞吐量成为瓶颈（详见 5.2.3 节）。

5.2.1 增量记录与压缩

当创建目录（mkdir）、删除目录（rmdir）等并发目录操作作用于同一目录时，操作间常会竞争目录属性元数据，进而触发事务中止，导致重试频繁且长尾延迟增加。在 CFS 系统 [49] 中，通过将每个操作拆分为两个单分片事务（一是插入 / 删除属性元数据，二是更新访问元数据与父目录属性元数据），可缓解 mkdir 与 rmdir 的竞争问题。但在 Mantle 中，这种双事务策略与 IndexNode 和 TafDB 之间的同步需求不兼容。

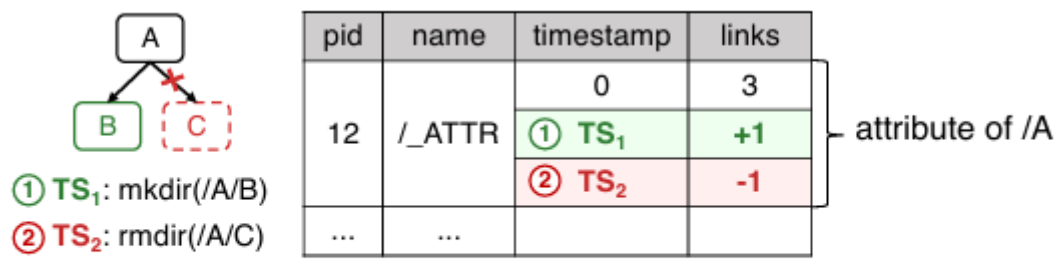


Figure 8. Using delta records avoids contention between different directory modifications within a directory.

为此，Mantle 引入增量记录（delta record）机制，通过替代原地更新来缓解竞争。如图 8 所示，每条增量记录由复合键标识，该复合键包含父目录 ID（pid）、特殊名称字段（“/_ATTR”）与事务时间戳（TS_{txn}）。操作不再直接更新主属性记录（时间戳为 0 的记录），而是插入一条唯一的增量记录。例如，在图 8 中，并发执行的“mkdir (/A/B)”与“rmdir (/A/C)”均需修改 / A 的属性元数据：若无增量记录，两者会竞争同一行数据；而有增量记录时，mkdir 操作会插入一条增量（TS₁，链接数 + 1），rmdir 操作插入另一条增量（TS₂，链接数 - 1），从而完全消除冲突。这些增量记录随后会被异步压缩到主目录属性元数据中，压缩过程中会通过共享锁确保目录元数据不被删除且保持完整。这种延迟更新策略在高并发场景下大幅减少了事务中止、重试与长尾延迟。

但该设计也存在权衡：目录状态查询（dirstat）操作需扫描增量记录才能得到准确结果。为解决这一问题，我们采用选择性启用策略——仅当目录持续存在竞争时才激活增量记录，从而在缓解竞争与保证查询准确性之间实现平衡。第 6.4 节的评估结果验证了这两项优化的性能收益。

5.2.2 优化跨目录重命名

Mantle 指定 IndexNode 作为跨目录重命名（dirrename）的协调者。IndexTable 中的每个目录元数据项均包含一个锁位（lock bit），使得代理可通过单 RPC 锁定重命名请求涉及的目录（如图 6 所示），无需通过多轮 RPC 获取分布式锁。

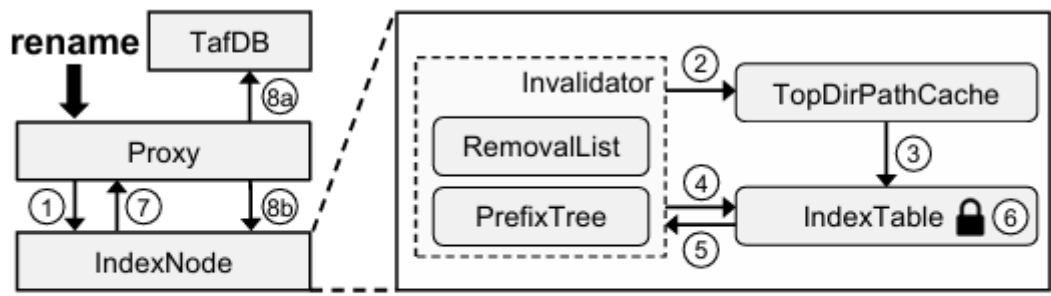


Figure 9. The workflows for processing directory rename operation

跨目录重命名的工作流程如图 9 所示，具体步骤如下：

1. 首先完成路径解析（步骤 1、2、3）；
2. IndexNode 将源目录路径加入 RemovalList（步骤 4），并通过设置 IndexTable 中源目录路径的锁位执行循环检测（步骤 5）；
3. 检查“源目录与目标目录的最近公共祖先到目标目录”路径上所有目录的锁状态（步骤 6）；
4. 将路径解析与循环检测结果返回给代理（步骤 7）：若检测到已有锁（表明与其他正在执行的重命名操作冲突），则中止当前操作并重试；若未发现冲突，代理通过分布式事务完成重命名——更新 MetaTable 与 IndexTable 中的所有相关元数据（步骤 8a、8b）；
5. 当 IndexTable 中源目录的访问元数据被删除时，重命名锁会自动释放。

5.2.3 提升 IndexNode 的 Raft 写性能

在高目录修改负载下，IndexNode 的写吞吐量常受限单 Raft 组的容量。这一限制主要源于数据持久化所需的频繁 fsync 操作——在高负载场景下，该操作会成为显著瓶颈。为缓解这一问题，Mantle 采用 Raft 批量提交机制，将多个操作聚合为单次提交。通过批量处理，Mantle 有效降低了高开销 fsync 调用的频率，减少磁盘 I/O 开销，从而在高负载下显著提升吞吐量。

5.3 容错机制

Mantle 的设计集成了强健的机制，以确保组件故障时元数据的一致性与高可用性。

对于元数据服务器故障，受影响的 Raft 组会自动触发主节点重选。Raft 协议本身可确保数据的完整性与可用性。

当代理节点无响应时，客户端会重新连接其他代理并提交未完成的请求。为保证操作的幂等性，每个元数据请求均携带客户端生成的唯一 UUID（通用唯一识别码）。如图 9 所示：

1. 若代理在步骤 1~7 期间故障，IndexNode 会重试这些步骤，并通过 UUID 识别原始请求是否已持有锁，避免死锁；
2. 若循环检测成功，新代理会继续执行步骤 8a/8b，完成两阶段提交（2PC）事务并释放锁；
3. 若故障发生在步骤 8a/8b 期间，标准的 2PC 恢复机制会确定最终提交结果，确保系统进入一致状态。